D2.7
Deployment of the PoC platform

Project Title	dealii-X: an Exascale Framework for Digital Twins of the Human Body
Project Number	101172493
Funding Program	European High-Performance Computing Joint Undertaking
Project start date	1 October 2024
Duration	27 months

dealii-X has received funding from the European High-Performance Computing Joint Undertaking Programme under grant agreement Nº 101172493

Deliverable title	D2.7: Deployment of the PoC platform
Deliverable number	D2.7
Deliverable version	1.0
Date of delivery	31/07/2026
Actual date of delivery	31/07/2026
Nature of deliverable	Report
Dissemination level	Public
Work Package	WP2
Partner responsible	eXact lab

Abstract	dealii-X D2.7 reports on the deployment of the first version of the Proof of Concept (PoC) platform, whose design was presented in D2.6. The platform, named DealiiX Platform, is accessed through a desktop application which provides a node-based low-code editor for building computational graphs, which are then executed by CORAL, a C++ computational graph engine backed by the deal.II library. The no-code layer envisioned in D2.6 is realized through subnetwork nodes, allowing users to encapsulate low-code graphs into reusable high-level components. The platform transparently targets computational resources ranging from the local machine to remote HPC systems managed by the Slurm scheduler, including MPI-parallel executions, and integrates a collaborative project server together with a web tool for the visualization of results and for the tagging of the meshes on which the simulated problem is defined. This deliverable describes the architecture and the main features of the deployed platform, its packaging, testing and continuous integration infrastructure, reports on the testing carried out with the dealii-X lighthouse applications and on the feedback collected from the early users, and outlines the activities planned for the final development phase of the project.
Keywords	PoC platform; low-code interface; no-code interface; computational graph; CORAL; deal.II; metascheduler; HPC

Document Control Information

Version	Date	Author	Changes Made
0.1	16/07/2026	M. Franzon	Initial draft
0.2	29/07/2026	P. Colturi, M. Franzon	Lighthouse testing; Outlook update
0.3	30/07/2026	P. Colturi, M. Franzon	VTK Manipulator rename; tagging
0.4	30/07/2026	P. Colturi, M. Franzon	Flow and consistency revision
0.5	30/07/2026	P. Colturi	Brain figures; float placement
1.0	31/07/2026	M. Kronbichler	Final edit

Approval Details

Approved by: Martin Kronbichler
Approval Date: 31/07/2026

Distribution List

- Project Coordinators (PCs)
- Work Package Leaders (WPLs)
- Steering Committee (SC)
- European Commission (EC)

Disclaimer: This project has received funding from the European Union. The views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European High-Performance Computing Joint Undertaking (the “granting authority”). Neither the European Union nor the granting authority can be held responsible for them.

Contents

1 Introduction

Deliverable D2.6 presented the design of the Proof of Concept (PoC) platform of the dealii-X project: a single environment in which professionals with different levels of technical expertise can build, run and inspect simulations based on the dealii-X libraries, without having to deal with the complexity of the traditional HPC workflow. The design identified four main requirements: a collaborative space, a modular architecture, an interaction model that allows deep customization without requiring specific coding skills, and a transparent usage of computational resources, from local machines to HPC clusters. These requirements led to the adoption of a combined low-code/no-code approach, in which users can define custom no-code nodes in terms of low-code logic.

The present deliverable reports on the deployment of the first version of the PoC platform, named *DealiiX Platform*. Following the development plan outlined in D2.6, the implementation activities of tasks 2.6.2 (development of the no-code/low-code graphical user interface) and 2.6.3 (development of a metascheduler system) started in M7 and proceeded iteratively, with a first version of the platform available at M17 and the deployment and testing phase (task 2.6.4) carried out between M18 and M22. The platform has been developed in the open, with an iterative release process: six versions have been released between M12 and M22, each accompanied by a public changelog, packaged installers for Linux and macOS, and an automated test suite.

The rest of the deliverable is organized as follows. Section 2 presents the architecture of the platform and its components. Section 3 describes the low-code node editor, the concrete realization of the design sketched in section 2 of D2.6. Section 4 describes the no-code layer, based on subnetwork nodes. Section 5 covers the execution modes and the metascheduling functionality, which transparently target local and remote computational resources. Section 6 presents the collaborative services and the visualization and mesh tagging tool. Section 7 reports on the testing of the platform with the dealii-X lighthouse applications and on the feedback collected during the deployment phase. Section 8 reports on packaging, deployment and quality assurance, and section 9 outlines the activities planned for the final phase of the project.

2 Architecture of the PoC platform

The PoC platform is composed of four cooperating components, shown in Fig. 1:

- **DealiiX Platform**, the desktop application that provides the user interface. It is built with the Electron framework and the Svelte 5 web framework, and embeds the node-based canvas editor on which the low-code and no-code interactions take place.
- **CORAL** (Computational Object-oriented Representation And Library), the C++ computational graph engine that executes the graphs built in the editor. CORAL exposes the classes and functions of an underlying scientific library as a *registry* of nodes, and executes *networks* of such nodes. The main backend is a deal.II plugin, which makes the deal.II finite element machinery available as graph nodes.
- **Coral Remote Server**, a REST API server written in Go that provides user authentication and collaborative project management, backed by an SQLite database.
- **Coral VTK Manipulator**, a Python web application for the visualization of simulation results in VTK formats and for the tagging of meshes with the identifiers used by deal.II, built on the Trame framework with a ParaView-based rendering backend.

The desktop application and CORAL communicate through two JSON-based protocols, which constitute the contract between the user interface and the computational engine:

- The **registry protocol** describes the node types made available by a CORAL backend: for each node, its arguments with their types, its input and output connections, and its nature (constructor, member function, free function, etc.). The registry is loaded by the application at startup, validated, and used to populate the palette of available nodes.
- The **network protocol** describes a computational graph as a list of nodes and typed edges. It is used to save and load projects, to exchange graphs between users, and to submit graphs to CORAL for execution.

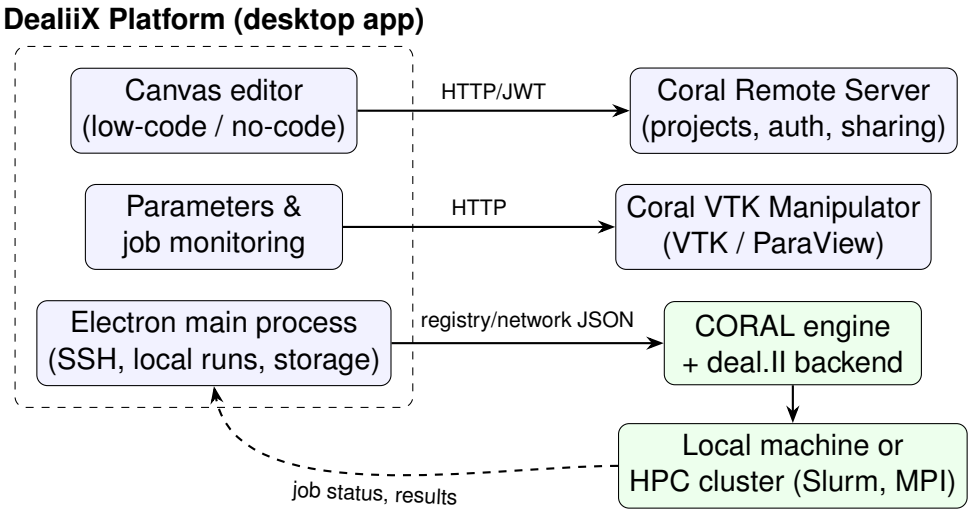


Figure 1: Architecture of the PoC platform and its components

This clean separation keeps the user interface completely agnostic of the underlying simulation library: any library wrapped by a CORAL backend automatically becomes available in the visual editor, without changes to the application. This fulfills one of the key modularity requirements set in D2.6.

3 The low-code node editor

The low-code interface implements the node-based programming model designed in section 2 of D2.6. The minimal set of object-oriented entities identified there is mapped onto a family of node types: elementary constructors for primitive values (integers, floating point numbers, booleans, strings), class constructors, member functions (including `const` and value-returning methods), free functions, and abstract base classes used for polymorphic connections. Each node exposes typed input and output pins, and edges between pins are validated in real time against the type system: a connection is accepted only if the output type of the source node matches the input type of the target node, including inheritance relations through abstract base classes. The same validation is applied when a graph is imported from a file or loaded from the remote server, so that only well-formed graphs reach the execution engine.

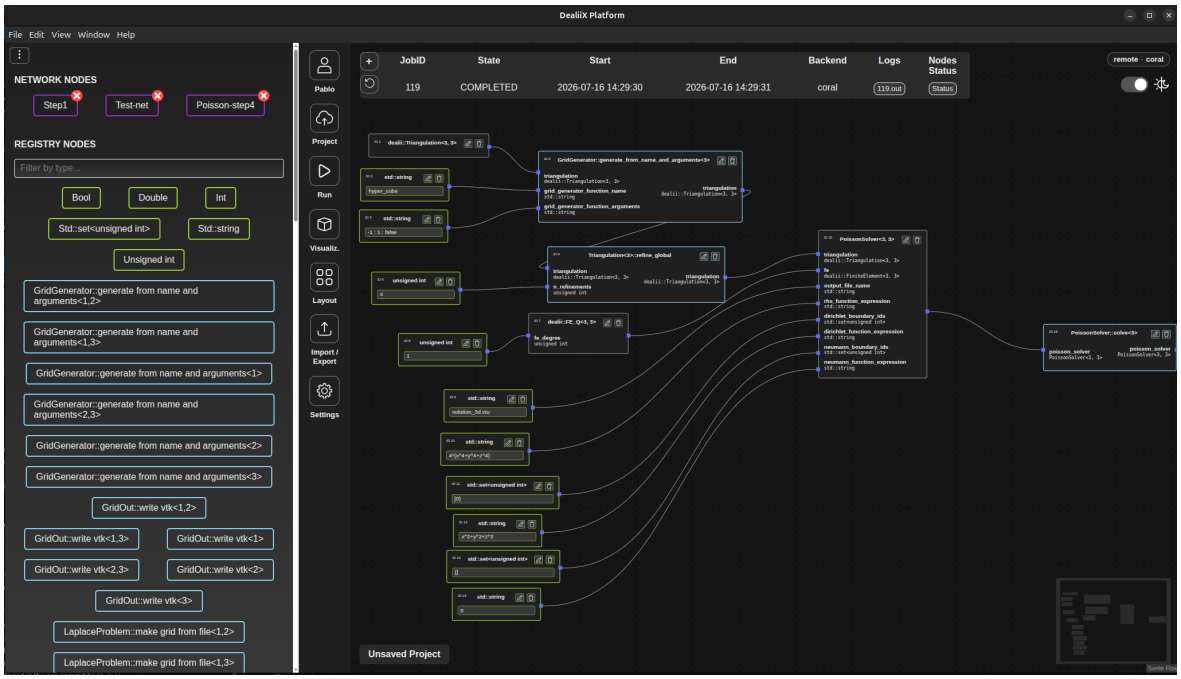


Figure 2: A computational graph built in the DealiiX Platform's node-based low-code editor, executed by CORAL and equivalent to deal.II's step-4 tutorial in three dimensions

On top of this typed graph model, the editor (Fig. 2) provides the interaction features expected from a modern visual programming environment:

- a searchable node palette, from which nodes are added to the canvas via drag and drop;
- a create-on-connect flow: dragging a connection onto an empty area of the canvas offers the compatible node types and creates the selected one, already connected;
- editable literal fields directly on primitive nodes, editable node names, multi-selection with bulk deletion;
- per-level undo/redo history and automatic graph layout based on a rank-based algorithm, with horizontal and vertical orientations;
- import and export of graphs as JSON files following the network protocol, so that graphs can be shared and versioned outside the platform;
- a light and a dark theme, with a fully semantic color system.

Simulation parameters are handled through a dedicated collapsible panel. The platform *probes* the simulation code to obtain its parameter tree: following the deal.II parameter handling conventions, both JSON and `.prm` text formats are supported, with automatic format detection. Parameters can be edited, sections can be duplicated (for parameter studies of repeated subsections), and the resulting file is uploaded alongside the graph at execution time.

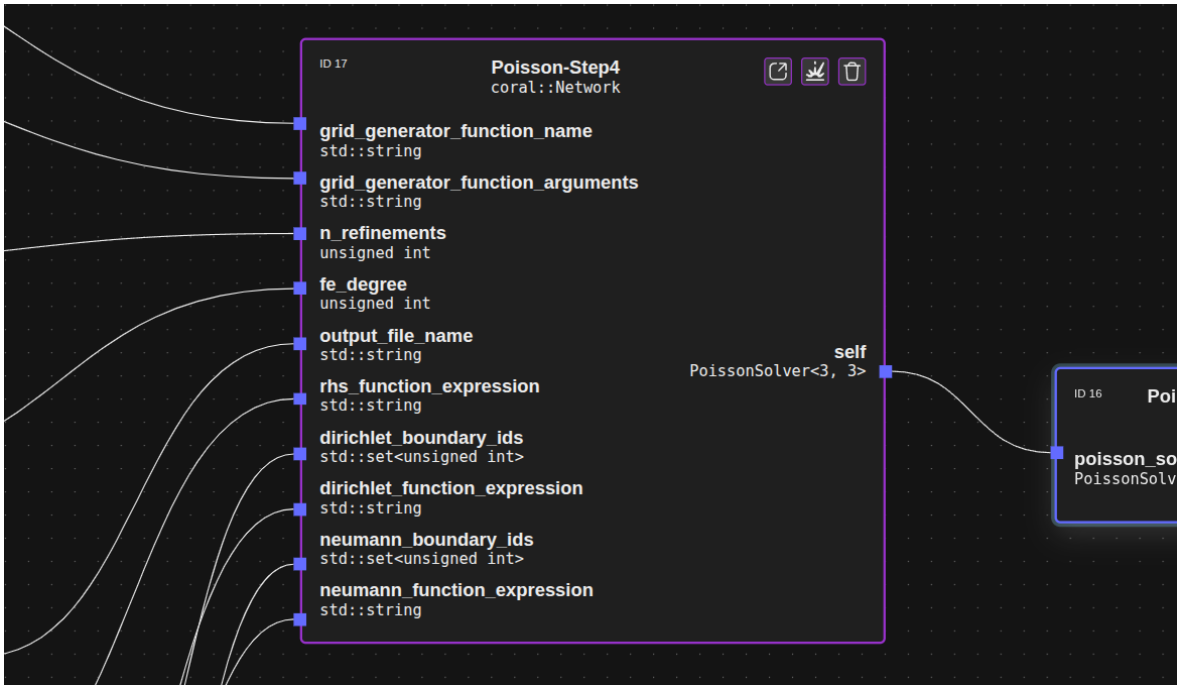
4 The no-code layer: subnetwork nodes

The key feature linking the low-code and the no-code approaches, as anticipated in D2.6, is the ability to encapsulate a collection of nodes into a single reusable node. In the deployed platform this is realized by *subnetwork nodes* (network nodes in the protocol nomenclature).

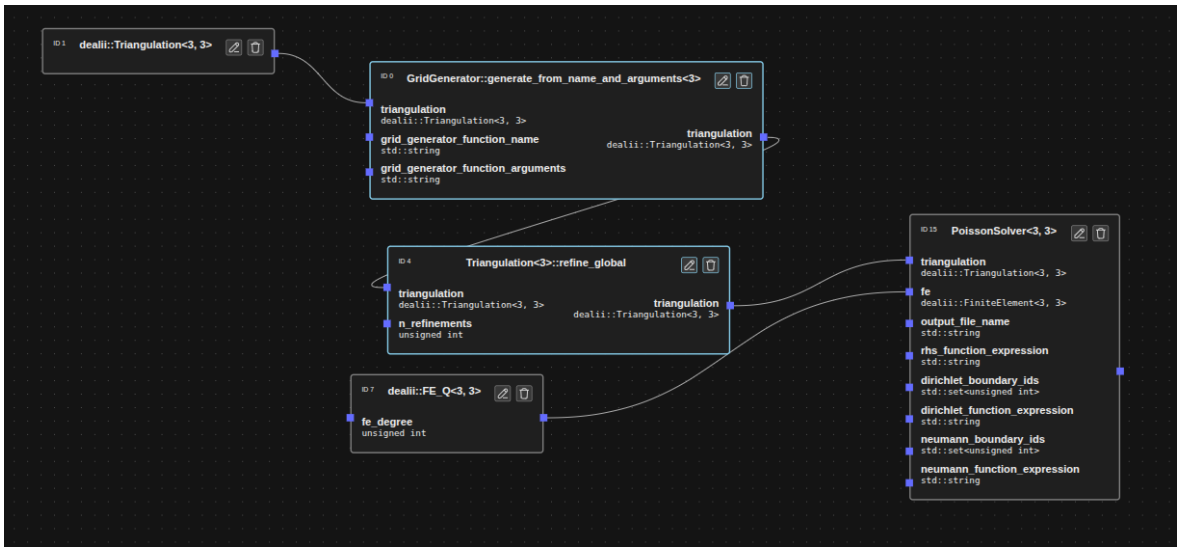
Any selection of nodes on the canvas can be collapsed into a named subnetwork node. The platform automatically computes the boundary of the selection: the unconnected inputs and outputs of the selected nodes become the external interface of the new node, while the internal edges are hidden. The resulting node behaves like any other node in the palette — it can be dragged onto the canvas, connected, and shared — but it encapsulates an entire computational graph, which is stored in its definition following the network protocol and persisted across sessions.

Subnetworks are fully editable in place: the user can enter a subnetwork node and modify its internal graph, navigating arbitrarily nested levels through a breadcrumb bar (Fig. 3). A subnetwork can also be *exploded* back into its constituent nodes in the parent graph. On export, every node is annotated with a qualified identifier that encodes its position in the nesting hierarchy, which is what allows the per-node execution monitoring described in Sec. 5 to work transparently across nested graphs.

This mechanism directly realizes the vision of D2.6: an expert user builds a simulation workflow with the low-code editor and collapses it into a single opaque node — for example a complete solver that only exposes a mesh input and a result output — and a non-expert user then operates purely at the no-code level, connecting a few high-level nodes and running the simulation without ever seeing the underlying logic.



(a) a collapsed high-level node



(b) the internal lower-level definition of the subnetwork node shown in (a)

Figure 3: Subnetwork nodes

5 Execution modes and metascheduling

The design requirement of a transparent usage of computational resources is implemented by a metascheduling layer that supports four execution modes, selected in the application settings and always visible through a dedicated execution badge:

- **Local + CORAL**: the graph is executed by a CORAL binary running directly on the user's machine;
- **Remote + CORAL**: the graph is submitted over SSH to a remote system, where it is queued through the Slurm workload manager and executed by CORAL;
- **Local + Executable**: any deal.II program following the DealiiX executable contract is run locally;
- **Remote + Executable**: the same, on a remote machine over SSH.

The *executable contract* is a deliberately minimal convention that makes existing deal.II applications usable from the platform without modification of the platform itself: when invoked with a parameters file path that does not exist, the program writes a template file containing all its parameters and exits (this is the probing step used to build the parameters panel); when the file exists, the program reads it and runs the simulation. The standard deal.II tutorial step-70 is included in the repository as a reference example and works out of the box.

For the remote modes, the platform manages the whole job lifecycle. Graph and parameter files are uploaded over SSH (using public key authentication), and a Slurm batch script is generated from a template. Two templates are provided, for serial and MPI execution; when MPI is enabled, the job configuration dialog exposes the number of nodes and tasks per node, and the exported network JSON is annotated with the MPI plugin block required by CORAL for parallel initialization. Submitted jobs are tracked in a persistent jobs table (Fig. 4) showing scheduler status (pending, running, completed, failed, cancelled), submission and completion times, and the backend used. For CORAL jobs, the execution status of every individual node of the graph — including nodes nested inside subnetworks, via their qualified identifiers — can be inspected from the interface, and the standard output of the job can be opened directly in the application.

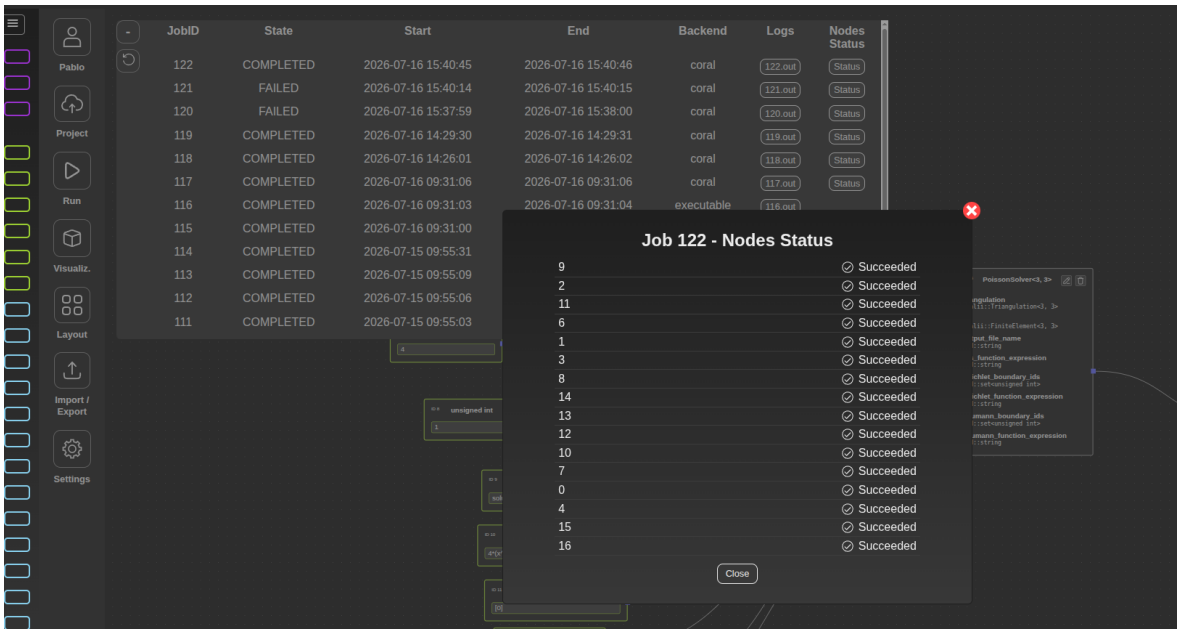


Figure 4: Job monitoring: jobs table and per-node execution status

Two deployment scenarios have been exercised during the testing phase. The first is a containerized environment, distributed with the platform, in which a Docker container runs CORAL together with a complete Slurm installation and an SSH server; this reproduces the interaction with a remote batch-scheduled cluster faithfully enough to develop and test the whole submission and monitoring path on a single machine. The second is an HPC cluster, on which the same containers have been deployed and reached over SSH, with the procedure documented step by step in the platform repository: `https://github.com/2listic/dealiiX-platform/blob/main/docs/remote-setup.md`. Since the platform assumes only SSH access and a Slurm scheduler, the same mechanism applies to other production HPC systems.

The layer described so far executes a single computational unit (a graph or an executable) per submission. Its natural extension is the orchestration of several such units, and this is the purpose of the pipeline mode described in Sec. 7. At the time of writing the orchestrator has been designed but not yet developed: the design defines the pipeline stages, the ordering edges between them and the translation of the resulting graph into a chain of dependent scheduler jobs, while the implementation is deliberately left to the final phase of the project, so that it can be shaped by the feedback of the users who will operate it.

6 Collaborative services, visualization and mesh tagging

The collaborative space requirement is met by the Coral Remote Server, a lightweight REST service with JWT-based authentication. From the application, users can register and log in, create projects, and save and load computational graphs to and from the server. Projects can be renamed, described, deleted, and — crucially for the collaborative workflows envisioned in D2.6 — shared with other users with read or write permissions. This allows, for instance, an expert user to prepare and share a validated simulation workflow that a colleague can open and run without rebuilding it.

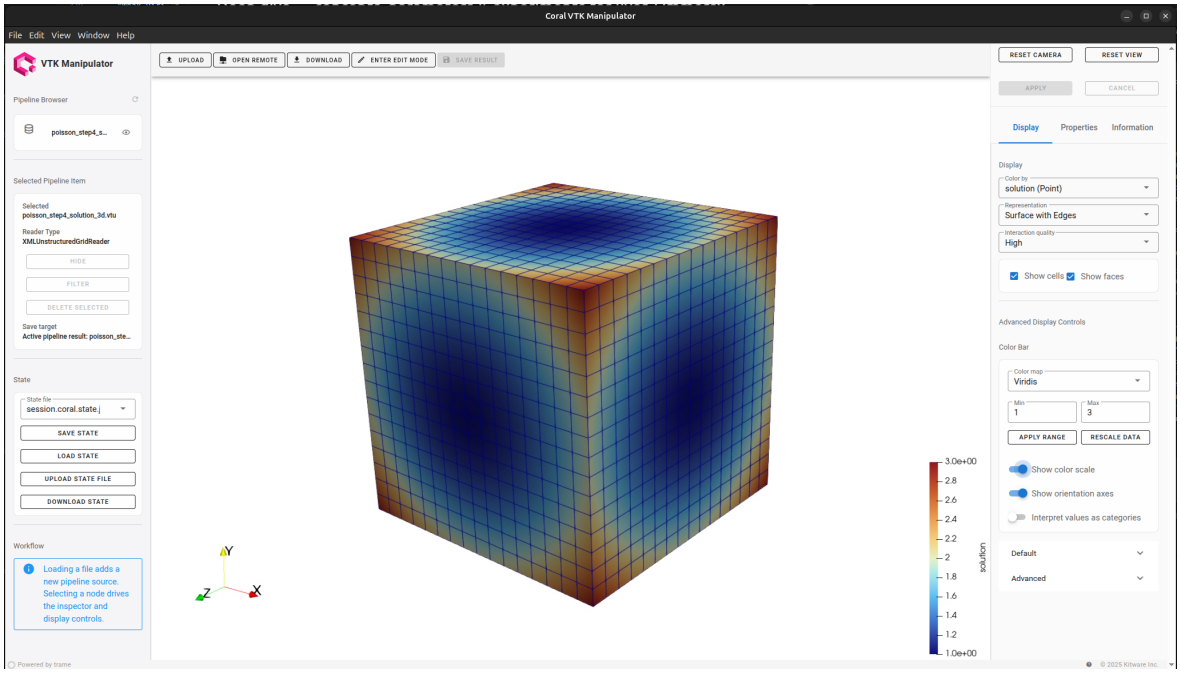


Figure 5: Poisson solution in the Coral VTK Manipulator as a result of the workflow of Fig. 2

Simulation results are inspected through the Coral VTK Manipulator (Fig. 5), a web-based application for VTK/deal.II formats (`.vtk`, `.vtu`), reachable directly from the desktop application. During the reporting period it has been substantially extended with a ParaView-based rendering backend providing filters, visualization pipelines and solution field rendering.

Beyond inspecting results, the tool has acquired mesh editing capabilities that make it useful as a pre-processing step and not only as a post-processing one.

Within an edit session both volume and surface cells can be picked interactively, the selection can be grown according to a feature angle, and an integer tag can be assigned to the whole selection. The tags are the ones that deal.II reads back from a VTK mesh, `MaterialID` and `ManifoldID`, and the meaning of the first depends on the dimension of the entity it is attached to: on a volume cell the material id identifies the material, and therefore the coefficients or the constitutive law used there, whereas on a surface cell the same number acts as the boundary indicator to which boundary conditions are applied. The manifold id, in turn, associates cells and faces with a *Manifold* object describing the exact geometry, so that the vertices created when the mesh is refined, and the higher-order mappings used in the assembly, follow the true curved geometry instead of the piecewise-linear approximation given by the coarse mesh. Tagging a mesh in this way therefore defines part of the problem that the simulation will solve, and it can be done from the interface without writing any code. The tool has been exercised both on the organ-scale meshes targeted by the project, such as the brain mesh of Fig. 6, and on reference meshes of several million cells used to characterize the performance of the editing session.

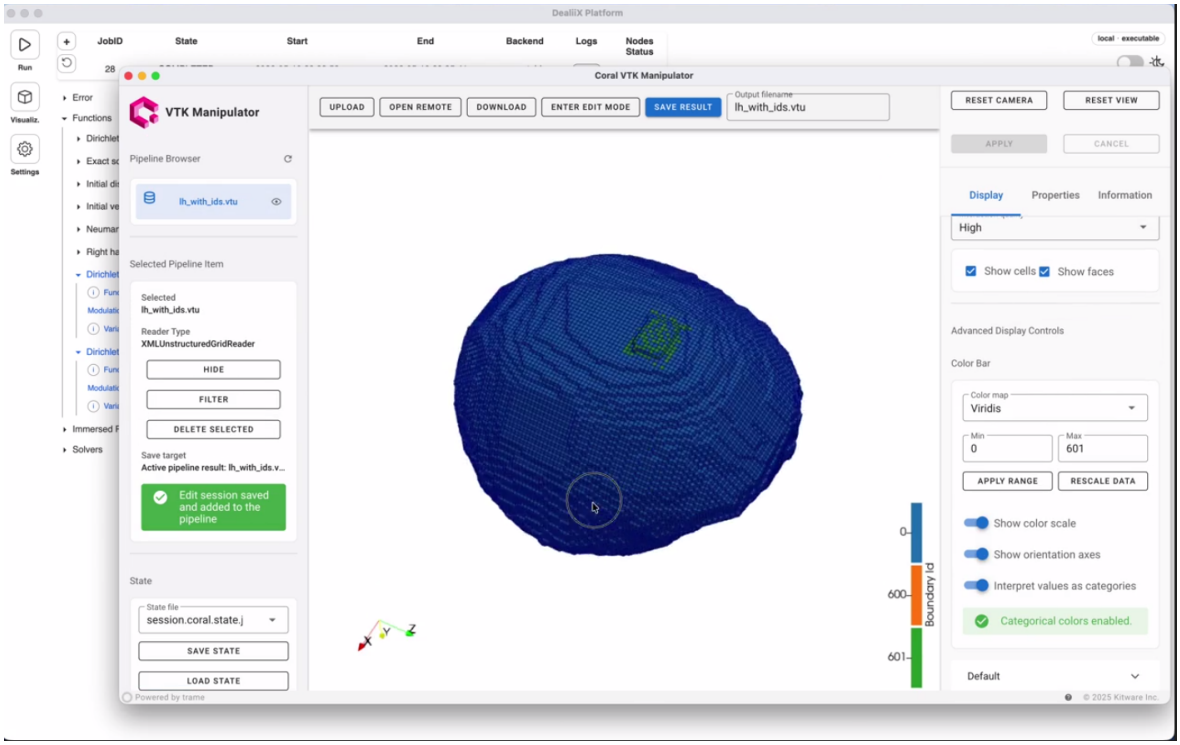


Figure 6: An edit session on the brain mesh opened in the Coral VTK Manipulator, where a group of surface cells has been given its own integer identifier

What the tool adds is the combination of two capabilities in one place. Tags are normally assigned when the mesh is generated: in a mesh generator such as

Gmsh they are attached to the surfaces and volumes of the geometry, before any cell exists. Organ meshes reconstructed from medical images have no such geometry, so the tags must be written onto the cells themselves, through a loop over them in the application code or in a conversion script; such loops are easy to get wrong and give no visual confirmation. ParaView, conversely, offers the filters and the capacity for large datasets, but no way to assign a value to a picked selection. The manipulator does both, so the tags can be assigned and checked where the mesh is already being inspected.

7 Testing with the lighthouse applications and early-user feedback

A central goal of the PoC platform is that a user without deep knowledge of the underlying simulation codes, and without the ability to write deal.II code or to assemble a computational graph, should still be able to configure and run a complete simulation. The executable mode described in Sec. 5 realizes this goal directly: given a program that follows the executable contract, the platform probes it for its parameter tree, presents that tree in the parameters panel in either JSON or `.prm` form, and dispatches the run to the selected local or remote target through the metascheduling layer, with monitoring and output retrieval available from the same interface. The interaction is reduced to editing parameters and pressing run.

This path has been tested on two of the dealii-X lighthouse applications, the brain simulator of FAU and the liver simulator of FVB-WIAS, and on the deal.II tutorial program step-70, which is included in the repository as a reference example of the contract. Because the contract requires no platform-side development per application, support in executable mode is a property of the mechanism rather than of a per-application integration effort: the heart application of POLIMI (`.prm` input) and the lungs application of TUM (JSON input) comply with the same parameter-file convention and are expected to work without changes to the platform. Fig. 7 shows the parameter tree of the brain application as the platform presents it in this mode.

These runs also show that the Coral VTK Manipulator of Sec. 6 is not an accessory to the workflow but one of its first stages. The grid is the first input that has to be settled: it is prepared and tagged in the manipulator, and its path is then one

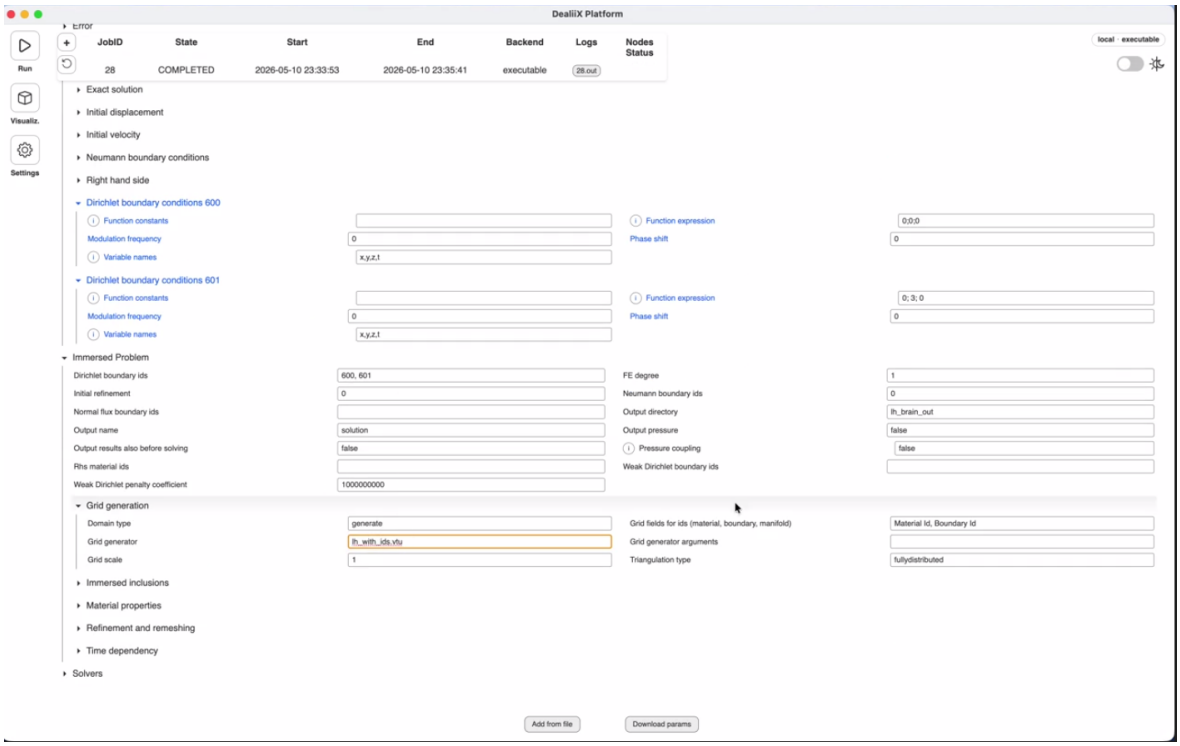


Figure 7: The parameter tree of the brain lighthouse application, presented in the parameters panel in executable mode. The grid prepared upstream (Fig. 6) is named in the grid generation entries, and the identifiers it carries reappear as the boundary ids on which the boundary conditions are set

of the entries of the parameter file that the executable reads, alongside the physical and numerical parameters. The identifiers assigned to the mesh are consumed in the same file, as the boundary ids to which the boundary conditions are attached, so that the upstream tagging and the parameters of the run describe one and the same problem. The same tool therefore takes part twice in a single simulation, upstream, where tagging the mesh defines the geometry, the materials and the boundaries of the problem to be solved (Fig. 6), and downstream, where the results are inspected once the run has completed; Fig. 8 shows the brain application at that second stage.

Testing also showed that the executable mode and the node editor should not remain two disjoint ways of using the platform. The observation that emerged most clearly is that an executable is itself a natural unit of composition (a black box with parameters, a run and an output), and therefore a natural candidate to become a node. This has led to the design of a *pipeline mode*, currently under development, in which a higher-level directed acyclic graph is built whose nodes are entire computational units: either a complete CORAL graph or a standalone executable. Edges express execution order, and the resulting graph is topologically sorted and sub-

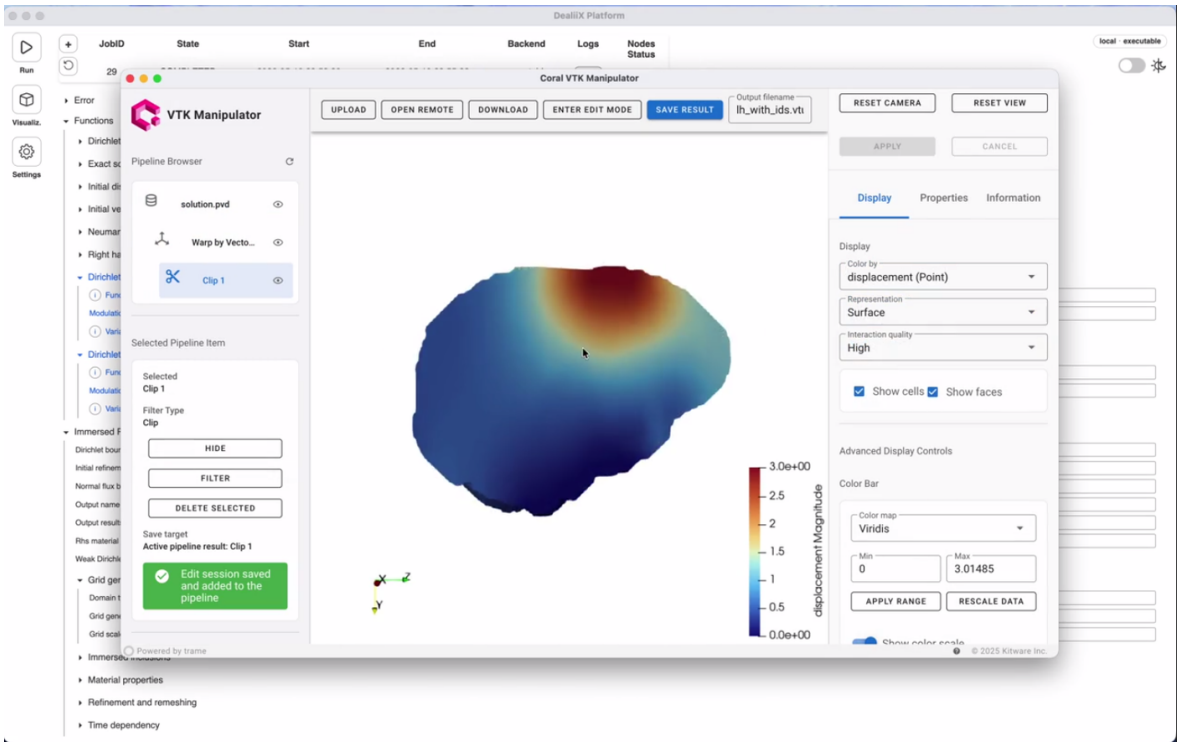


Figure 8: Results of the brain lighthouse application, inspected in the Coral VTK Manipulator

mitted to the scheduler as a chain of dependent jobs, giving the metascheduler an orchestration capability that neither CORAL nor a single executable provides on its own. Pipeline mode is a node editor in its own right, built on the same canvas technology as the low-code editor, and is therefore positioned to reuse the interaction features already available there. In this way a lighthouse application driven in executable mode ceases to be an alternative to the graph paradigm and becomes a component within it.

Feedback collected from project partners during the testing phase was consistently positive on the central proposition of D2.6, and converged with the development team's own findings. It concerned both the platform and the Coral VTK Manipulator, and ranged from the ergonomics of the tagging workflow to the behaviour of the tools on the deployed environments; the improvements it calls for are taken up as the activities of the final phase, listed in Sec. 9.

8 Packaging, deployment and quality assurance

The platform is distributed as native installers for Linux (`.deb`) and macOS (`.dmg`), produced by an automated release pipeline: tagging a version on the main branch triggers GitHub Actions workflows that run the full check and test suite, build the application, and attach the installers to a public GitHub release. For evaluation and testing, the full server-side stack (CORAL with Slurm and SSH, the remote server, and the VTK manipulator) is started with a single `docker compose up` command.

Six versions have been released during the reporting period, following the iterative plan of D2.6:

- **1.0.9** (September 2025): first packaged version — canvas editor with typed validation, deal.II nodes, SSH execution, deployment on real remote machines;
- **1.1.0** (January 2026): remote server integration (authentication, projects, sharing), jobs panel, protocol simplification;
- **1.2.0** (January 2026): network (subnetwork) nodes, graph download, unit testing infrastructure;
- **1.3.0** (March 2026): MPI support, persistent jobs with per-node execution status, job log inspection, parameter handling;
- **1.4.0** (May 2026): execution modes with local runs and the executable contract, in-place subnetwork editing, full TypeScript migration, redesigned settings;
- **1.4.1** (July 2026): single-command full stack, end-to-end test suites, complete theming overhaul, `.prm` parameter support.

Quality assurance is enforced at three levels. At commit time, git hooks run the linter, the type checker and the code formatter. On every push and pull request, a continuous integration pipeline runs static type checking of both the frontend and the Electron backend, plus the unit test suite (Vitest), and blocks merging on failure. Finally, two tiers of end-to-end tests, based on Playwright driving the real Electron application, cover the interactive flows: Tier 1 exercises the canvas (node creation, connections, undo/redo, graph import, subnetwork collapse and explode) without

external services, and runs headlessly in CI on every push; Tier 2 covers authentication and project management flows against a live Coral Remote Server instance. Both tiers run against an isolated configuration store, so tests are reproducible and cannot interfere with a user's installation.

9 Outlook

With the deployment of the first version of the PoC platform, task 2.6.4 of the development plan presented in D2.6, reproduced in Fig. 9, is completed. The final phase of the project (task 2.6.5, M23–M27) is devoted to the development of additional features, driven by the feedback collected from the project partners during the testing period. The main directions are:

- enrichment of the node registries with the dealii-X application libraries developed in the other work packages, moving from tutorial examples towards the organ-scale digital twin workflows targeted by the project;
- development of the orchestrator designed during this phase: passing of artifacts between pipeline stages, verification that the expected artifacts of a stage exist before the stages depending on it are released, and a short path to edit the parameters of an individual stage of a pipeline directly from the orchestrator canvas;
- MPI support in executable mode;
- improvements to the Coral VTK Manipulator: a clearer visual feedback in the selection and assignment of cells, a more systematic testing of the visualization filters, an assessment of the responsiveness of the editing session on large meshes in the deployed environments, and a verification of the formats in which a tagged grid is best saved for consumption by the applications;
- continued improvement of the no-code user experience for non-expert users.

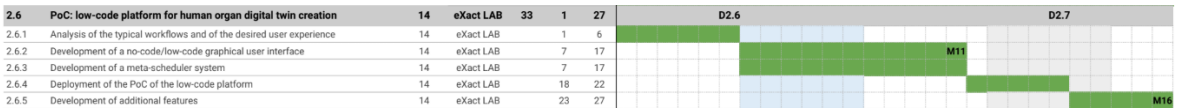


Figure 9: Timeline of the activities of task 2.6; D2.7 marks the end of the deployment phase (task 2.6.4)

The PoC platform is one of the main exploitable results of the dealii-X project: it demonstrates, end to end, that the simulation capabilities developed in the project can be operated through an intuitive low-code/no-code environment by users with widely different technical backgrounds, and it provides the foundation on which the final version of the platform will be delivered at the end of the project.